

Contents

CCSDS 235.1 & LSIS-AFS Mid-Project Workshop	1
Logistics	1
Prerequisites	2
Day 1 — Progress Checkpoint, Standards Clarification & GitLab Onboarding	2
09:15 — Welcome and Workshop Objectives	2
09:30 — GitLab Onboarding & Initial Submissions	2
10:15 — Team Progress Checkpoint	2
11:00 — Coffee Break	2
11:15 — Standards Clarification Sessions	2
12:30 — Lunch	3
13:30 — Technical Mentoring & Development Time	3
15:00 — Guided Tour of Goonhilly Earth Station	3
16:00 — Informal Demonstrations & CI Review	3
17:15 — Day 1 Wrap-Up	3
Day 2 — Interoperability Testing	3
09:15 — Interoperability Test Briefing	3
09:30 — CCSDS 235.1: Phase 1 — SPDU Wire Format Exchange	3
09:30 — LSIS-AFS: Level 0 & Level 1 (parallel)	4
10:30 — Coffee Break	4
10:45 — CCSDS 235.1: Phase 2 — COP-P Behaviour Verification	4
10:45 — LSIS-AFS: Level 2 — Signal File Exchange (parallel)	5
11:30 — CCSDS 235.1: Phase 3 — Live Protocol Exchange	5
11:30 — LSIS-AFS: Level 3 — Cross-Decode (parallel)	5
12:30 — Lunch	5
13:30 — Interoperability Results Review	5
14:30 — Final Project Phase Planning	5
15:15 — Support & Coordination	6
16:00 — Workshop Close	6
GitLab Interop Infrastructure	6
Repository Structure	6
CI Pipeline	6
Team Onboarding (Day 1, 09:30)	7
Partial Participation Guide	7
Reference	7
CCSDS 235.1 Specification Sections	7
LSIS-AFS Key References	7

CCSDS 235.1 & LSIS-AFS Mid-Project Workshop

Hosted by Goonhilly Earth Station

Late June / Early July 2026

Logistics

- **Location:** Goonhilly Earth Station, near Helston, Cornwall (Lizard Peninsula)

- **Transport:** Car transport essential
 - **Teams:** 10
 - **Infrastructure:** Local macOS GitLab server for automated interoperability testing
 - **Contact:** David Johnson, G4DPZ, Hon Sec, AMSAT-UK — esa-competition@amsat-uk.org — 07733 106990
-

Prerequisites

Each team attending should bring:

For CCSDS 235.1: - A working SPDU encoder/decoder (all types) - A working COP-P implementation (FOP-P and FARM-P) - A working Frame Layer (P-frames and U-frames) - The ability to produce and consume raw byte arrays (hex dumps)

For LSIS-AFS: - A CLI tool or Docker container conforming to the CLI contract - At minimum: spreading code generation (`generate-codes`) - Ideally: frame encoding (`encode --format frame`) and/or signal generation (`encode --format iq32`)

General: - A laptop capable of running your implementation - Git client configured (for pushing to the local GitLab server)

Day 1 — Progress Checkpoint, Standards Clarification & GitLab Onboarding

09:15 — Welcome and Workshop Objectives

Overview of goals, workshop structure, and introduction to the GitLab-based interop testing infrastructure.

09:30 — GitLab Onboarding & Initial Submissions

Hands-on session to get all teams connected: - Each team clones the interop repository from the local GitLab server - Push initial artifacts (CCSDS 235.1 hex dumps, LSIS-AFS CLI outputs) - First CI pipeline run — immediate feedback on reference comparison - Resolve any tooling or connectivity issues

10:15 — Team Progress Checkpoint

Brief high-level updates from each team covering: - Implemented protocol features (Gateways 1–3 expected complete, Gateway 4 in progress) - Architecture and language choices - Challenges encountered - Readiness for interoperability testing

11:00 — Coffee Break

11:15 — Standards Clarification Sessions

Parallel tracks focusing on specification areas most likely to cause interop failures:

Track A: LSIS-AFS (Lunar Signal-in-Space — Augmented Forward Service) - Gold code generation (polynomials, shift direction, init vectors) - Frame structure (sync pattern, subframe

layout, symbol encoding) - LDPC/BCH encoding details - Signal format (sample rate, float32 layout, BPSK mapping convention)

Track B: CCSDS 235.1 — Space Communications Session Control - SPDU Field Packing (Section 3): bit-field layout, byte ordering, MSB-first convention - Modulo-256 Arithmetic (Section 6.1): subtraction and comparison semantics - COP-P Retransmission (Section 6.2): go-back-N, R(R)-triggered vs gap-triggered - Window Management (Section 6.2, Annex F): max window 127, wrap-around handling - Expedited vs Sequence Controlled QoS (Section 6): independent counters, FARM-P rules - Persistence Mechanism (Section 4): WAITING_PERIOD, RESPONSE, LIFE-TIME - Session State Transitions (Section 5): state table interpretation

12:30 — Lunch

13:30 — Technical Mentoring & Development Time

Private consultation sessions with protocol experts and GNU Radio mentors: - Interpreting specification details - Debugging protocol behaviour - Signal processing guidance

Teams not in consultation sessions may fix issues identified by the CI pipeline and re-push.

15:00 — Guided Tour of Goonhilly Earth Station

- Antenna systems
- Mission operations facilities
- RF systems used in deep-space communications

16:00 — Informal Demonstrations & CI Review

Teams may optionally demonstrate aspects of their implementations:

- **SPDU Round-Trip:** Encode a PLCW, produce hex dump, decode another team's hex dump
- **COP-P Delivery:** Send numbered frames through FOP-P/FARM-P, show retransmission
- **Frame Construction:** Display P-frame and U-frame wire formats
- **LSIS Signal Generation:** Show I/Q output, demonstrate decode

Review CI pipeline results from the day's submissions — discuss any surprises.

17:15 — Day 1 Wrap-Up

Summary of key clarification points. Preparation for Day 2 interop testing.

Day 2 — Interoperability Testing

09:15 — Interoperability Test Briefing

Overview of the day's structure. Confirm team pairings and rotation schedule.

09:30 — CCSDS 235.1: Phase 1 — SPDU Wire Format Exchange

Each team produces 5 encoded artifacts as hex dumps:

#	Artifact	Specification
1	Type F1 PLCW (2 bytes)	Report Value=127, Retransmit=false, PCID=0, Expedited Counter=3
2	Type F2 PLCW (4 bytes)	Report Value=500, Retransmit=true, PCID=1, Expedited Counter=6
3	Variable-Length SPDU	Type 1 (Directive), SET V(R) with SEQ_CTRL_FSN=42
4	P-frame	Version-3, containing artifact #1, Expedited QoS
5	U-frame	Version-3, 10-byte payload 0x00-0x09, Sequence Controlled QoS, seq=7

Teams push artifacts to GitLab. CI pipeline runs pairwise comparisons automatically.

09:30 — LSIS-AFS: Level 0 & Level 1 (parallel)

Level 0 — Spreading Code Validation (30 min) - Each team pushes `codes.txt` (210 lines × 512 hex chars) - CI runs `diff` across all teams pairwise - Mismatch debugging: compare first 24 chips against Annex3

Level 1 — Frame Encoding (30 min) - Canonical input: PRN=1, FID=0, TOI=42, WN=100, ITOW=250, CED=all zeros - Each team pushes `frame.bin` (exactly 6000 bytes) - CI runs `cmp` across all teams pairwise - On mismatch: CI reports first divergent byte offset

10:30 — Coffee Break

10:45 — CCSDS 235.1: Phase 2 — COP-P Behaviour Verification

Paired teams validate sequence number handling using agreed scenarios:

Scenario	Initial State	Input	Expected Result
Normal in-sequence	V(R)=10	N(S)=10	Accept, V(R)=11
Small gap	V(R)=10	N(S)=12	Gap detected, R(S)=true
Wrap-around in-sequence	V(R)=255	N(S)=255	Accept, V(R)=0
Wrap-around gap	V(R)=254	N(S)=1	Gap of 3, R(S)=true
Duplicate	V(R)=10	N(S)=8	Discard
PLCW ack across wrap	NN(R)=250	N(R)=3	Acknowledge 9 frames
Window check across wrap	NN(R)=250, V(S)=3	Window=10	9 outstanding, 1 slot free

Teams push scenario results as JSON. CI validates against expected outputs.

10:45 — LSIS-AFS: Level 2 — Signal File Exchange (parallel)

- Each team generates I/Q signal (~94 MB): `encode --format iq32 --rate 1023000`
- Files uploaded to GitLab LFS or shared network directory
- CI cross-decodes: every team's signal decoded by every other team's decoder
- Pass criterion: recovered FID, TOI, WN, ITOW match original values

11:30 — CCSDS 235.1: Phase 3 — Live Protocol Exchange

For teams with TCP/UDP transport: 1. One team as caller (FOP-P sender), other as responder (FARM-P receiver) 2. Send sequence-controlled frames over the network 3. Responder returns PLCWs 4. Introduce gaps — verify retransmission triggers and recovery 5. Verify end-to-end delivery

11:30 — LSIS-AFS: Level 3 — Cross-Decode (parallel)

Full round-trip with varied parameters:

Test	PRN	FID	TOI	WN	ITOW
1	1	0	0	0	0
2	1	0	42	100	250
3	5	2	99	8191	503
4	12	3	50	1000	100

CI generates full N×N encode/decode matrix across all teams.

12:30 — Lunch

13:30 — Interoperability Results Review

CI Dashboard Review: - Walk through the N×N compatibility matrices for both protocols - Identify clusters of agreement and disagreement - Highlight pairwise mismatches that suggest spec ambiguities vs implementation bugs

Discussion: - Successful protocol interactions - Byte-ordering or field-packing mismatches discovered - CRC algorithm discrepancies - Spec areas requiring clarification from the working group

Team Reports: Each team produces a brief compatibility report: - Which artifacts matched across teams - Which had discrepancies - Root cause analysis - Planned fix and timeline

14:30 — Final Project Phase Planning

Review of remaining milestones toward the 31 August deadline:

CCSDS 235.1: - Gateway 4: Physical Layer Abstraction - Gateway 5: State Machine (Section 5.1–5.3) - Gateway 6: Hailing & Session Control (Section 5.1.3, Section 4) - Gateways 7–10: Integration, Conformance, Interoperability, Documentation

LSIS-AFS: - Gateway 5: Decoding Infrastructure - Gateway 6: Message Parser - Gateway 7: Integration & Testing - Gateway 8: Documentation & Examples

15:15 — Support & Coordination

- Ongoing mentoring resources and communication channels
- Follow-up technical meetings schedule
- GitLab server access post-workshop (if available)
- Coordination for final interoperability testing in August

16:00 — Workshop Close

Summary of workshop outcomes and next steps.

GitLab Interop Infrastructure

Repository Structure

```
interop-workshop/  
  .gitlab-ci.yml  
  teams/  
    team-01/  
      scsc/                                ← CCSDS 235.1 artifacts  
        f1_plcw.bin + .json  
        f2_plcw.bin + .json  
        directive.bin + .json  
        pframe.bin + .json  
        uframe.bin + .json  
      lsis/                                ← LSIS-AFS outputs  
        codes.txt  
        frame.bin  
        signal.iq32 (LFS)  
    team-02/  
    ...  
    team-10/  
  scripts/  
    compare_spdu.sh  
    compare_codes.sh  
    compare_frames.sh  
    cross_decode.sh  
    validate_copp_scenarios.sh  
    generate_matrix.sh  
  results/  
    scsc_matrix.md  
    lsis_matrix.md
```

CI Pipeline

On every push: - Full N×N pairwise comparison across all teams - Binary diff for LSIS-AFS (byte-identical expected at code/frame level) - Field-value comparison for CCSDS 235.1 (decode +

check fields) - Cross-decode for LSIS-AFS signals - Generates compatibility matrix showing agreement/disagreement clusters - Immediate pass/fail feedback per team pair

Team Onboarding (Day 1, 09:30)

1. Connect to local WiFi / Ethernet
2. Clone: `git clone http://gitlab.local/interop-workshop/interop-workshop.git`
3. Create team directory: `teams/<team-name>/scsc/` and `teams/<team-name>/lsis/`
4. Push artifacts
5. Check pipeline results in GitLab CI/CD → Pipelines

Partial Participation Guide

Team capability	CCSDS 235.1 participation	LSIS-AFS participation
SPDU layer only	Phase 1 (artifact exchange)	—
SPDU + COP-P	Phases 1–2	—
Full CCSDS + TCP/UDP	Phases 1–3	—
Code generation only	—	Level 0
Encoder only	—	Levels 0–2 (generate for others)
Decoder only	—	Levels 2–3 (decode others' signals)
Full pipeline	—	All levels
Both protocols	All phases	All levels

Reference

CCSDS 235.1 Specification Sections

Topic	Section
SPDU Encoding/Decoding	Section 3
Persistence Mechanism	Section 4
Session State Tables	Section 5.1–5.3
Frame Sending/Receiving	Section 5.4–5.6
COP-P Overview	Section 2.3
Modulo-256 Arithmetic	Section 6.1
FOP-P (Sending Procedures)	Section 6.2
FARM-P (Receiving Procedures)	Section 6.3
MIB Parameters	Annex F
Vehicle Controller Notifications	Annex G

LSIS-AFS Key References

Topic	Reference
Spreading codes	Annex3 tables
BCH encoding	Generator polynomial 763
LDPC matrices	Annex1 CSV files
CRC-24 polynomial	Specification Section TBD
Frame structure	68 sync + 52 SB1 + 5880 interleaved
Signal format	Float32 LE interleaved I/Q, 1.023 MHz